

Systemy liczbowe II:
Arytmetyka binarna, kod U2 i liczby rzeczywiste
Matura rozszerzona z informatyki – Sesja 2

Kacper Gębusia

- 1 Powtórzenie wiadomości z poprzedniej sesji
- 2 Dodawanie i odejmowanie: binarne i szesnastkowe
- 3 Przesunięcia bitowe
- 4 Kod U2 – zapis liczb ujemnych
- 5 Porównanie ZM / U1 / U2
- 6 Przepiętnienie (overflow)
- 7 Liczby rzeczywiste w systemie dwójkowym
- 8 Budowa liczby zmiennoprzecinkowej
- 9 Ćwiczenia

Powtórzenie

- 1 Zamień na system o podstawie 5: $100_{10} = ?_5$
- 2 Zamień na system dziesiętny: $341_6 = ?_{10}$
- 3 Zamień na system o podstawie 20: $300_{10} = ?_{20}$
- 4 Uzupełnij kod w taki sposób, aby funkcja implementowała schemat Hornera (Zwróć uwagę na typy danych - pamiętaj, że nie można dodawać do siebie liczby i ciągu znaków):

```
def horner(liczba, podstawa):  
    wynik = 0
```

```
    for cyfra in liczba:
```

```
        
```

```
    return wynik
```

```
print(horner("1101", 2))
```

```
print(horner("210", 3))
```

Dodawanie binarne

Zasady: $0 + 0 = 0$, $0 + 1 = 1$, $1 + 1 = 10$ (0, przen. 1), $1 + 1 + 1 = 11$ (1, przen. 1).

Przykład 1: $1011_2 + 1101_2$

$$\begin{array}{rcccccc} & 1 & 0 & 1 & 1 & (11) \\ + & 1 & 1 & 0 & 1 & (13) \\ \hline 1 & 1 & 0 & 0 & 0 & (24) \end{array}$$

Wynik: **11000** (sprawdzenie: $11 + 13 = 24 \checkmark$)

Odejmowanie binarne

Zasady z pożyczką: $1 - 0 = 1$, $1 - 1 = 0$, $0 - 0 = 0$, $0 - 1 = 1$ (pożyczka z sąsiedniej pozycji).

Przykład 2: $1011_2 - 0101_2$

$$\begin{array}{rcccccl} & 1 & 0 & 1 & 1 & (11) \\ - & 0 & 1 & 0 & 1 & (5) \\ \hline & 0 & 1 & 1 & 0 & (6) \end{array}$$

Wynik: **0110** (sprawdzenie: $11 - 5 = 6 \checkmark$)

Kluczowa obserwacja: zasada jest identyczna w każdym systemie pozycyjnym – zmienia się tylko próg przeniesienia/pożyczki (dwójkowy: 2, ósemkowy: 8, szesnastkowy: **16**).

Przykład 3 (dodawanie): $3A_{16} + 2F_{16}$

$$\begin{array}{r} 3A \ (58) \\ +2F \ (47) \\ \hline 69 \ (105) \end{array}$$

$A(10) + F(15) = 25 \geq 16 \Rightarrow$ zapis 9, przeniesienie 1; $3 + 2 + 1 = 6$. Wynik: **69**₁₆

Odejmowanie w systemie szesnastkowym

Przykład 4: $5C_{16} - 2F_{16}$

$$\begin{array}{r} 5C \ (92) \\ -2F \ (47) \\ \hline 2D \ (45) \end{array}$$

$C(12) - F(15)$: $12 < 15 \Rightarrow$ pożyczka: $12 + 16 - 15 = 13$ (D); $5 - 2 - 1 = 2$. Wynik: **$2D_{16}$**

Przesunięcie bitowe – mnożenie i dzielenie przez 2

- **Przesunięcie w lewo o 1 bit** (dopisanie 0 na końcu) = **mnożenie przez 2**
- **Przesunięcie w prawo o 1 bit** (odcięcie ostatniego bitu) = **dzielenie całkowitoliczbowe przez 2** (odcięty bit = reszta)
- Przesunięcie o n pozycji $\Rightarrow$ mnożenie/dzielenie przez 2^n
- W Pythonie: operatory $\ll$ i $\gg$

Przykład 5: $1011_2 = 11_{10}$

- $1011 \ll 1 = 10110_2 = 22_{10}$ (bo $11 \cdot 2 = 22$)
- $1011 \gg 1 = 101_2 = 5_{10}$ (bo $11 \div 2 = 5$ reszty 1)

Kod uzupełnień do dwóch (U2)

Po co? Komputer nie ma osobnego "znaku minus- liczby ujemne trzeba zapisać samymi bitami 0/1.

Zasady:

- Na n bitach w U2 mieszczą się liczby z zakresu $[-2^{n-1}, 2^{n-1} - 1]$ (8 bit: $-128 \dots 127$)
- Liczby **dodatnie** zapisujemy zwyczajnie (najstarszy bit = 0)
- Liczbę **ujemną** $-x$: zapisz $|x|$ binarnie $\rightarrow$ **zneguj bity** $\rightarrow$ **dodaj 1**
- Najstarszy bit = 1 $\Rightarrow$ liczba ujemna

Przykłady – zapis i odczyt w U2

Przykład 6: Zapisz -5 w U2 na 8 bitach.

$$\begin{array}{rcll} 5 & = & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ \text{negacja} & = & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 \\ +1 & = & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 \quad \leftarrow \text{to jest } -5 \text{ w U2} \end{array}$$

Przykład 7: Jaką wartość ma 10000001_2 w U2 (8 bit)?

$$\begin{array}{rcll} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & (\text{najstarszy bit}=1 \Rightarrow \text{ujemna}) \\ \text{negacja} & = & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ +1 & = & 0 & 1 & 1 & 1 & 1 & 1 & 1 = 127 \end{array}$$

Wynik: **-127**

Trzy sposoby zapisu liczb ujemnych

- **ZM (znak-moduł):** najstarszy bit to znak, reszta to wartość bezwzględna
- **U1 (uzupełnienie do 1):** negujemy wszystkie bity liczby dodatniej (bez $+1$)
- **U2:** jak U1, ale dodatkowo $+1$

Porównanie ZM / U1 / U2 (8 bitów)

Cecha	ZM	U1	U2
Zapis -5	10000101	11111010	11111011
Zakres	$-127 \dots +127$	$-127 \dots +127$	$-128 \dots +127$
Zapisy zera	dwa	dwa	jeden
Dodawanie	osobna logika	korekta na końcu	działa wprost

Problem podwójnego zera: w ZM i U1 $+0$ i -0 to dwa różne zapisy tej samej wartości – to komplikuje sprzęt i "psuje" symetrię zakresu. W U2 zero ma tylko jeden zapis – dlatego to standard we współczesnych komputerach.

Dodawanie w U2 – dlaczego jest wygodne

Przykład 8: $5 + (-3)$ na 8 bitach.

	0	0	0	0	0	1	0	1	(5)
+	1	1	1	1	1	1	0	1	(-3 w U2)
1	0	0	0	0	0	0	1	0	obcinamy nadmiarowy bit poza rejestrem
	0	0	0	0	0	0	1	0	= 2

Wynik: **2** (sprawdzenie: $5 - 3 = 2$ ✓) – dodawanie działa "samo", bez osobnej logiki dla odejmowania.

Przepiętnienie (overflow)

Liczby ze znakiem na n bitach mają ograniczony zakres. Gdy wynik działania wykracza poza ten zakres – mamy **przepiętnienie**.

Reguła: jeśli dodajemy dwie liczby o **tym samym znaku**, a wynik ma **przeciwny znak** – doszło do przepiętnienia.

Przykład przepełnienia

Przykład 9: $100 + 50$ na 8 bitach (U2)

$100_{10} = 01100100_2$, $50_{10} = 00110010_2$

$$\begin{array}{rcccccccc} & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & (100) \\ + & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & (50) \\ \hline & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & (150) \end{array}$$

Matematycznie $100 + 50 = 150$, ale zakres 8-bit. U2 to tylko $-128 \dots 127$. Bit znaku wyniku $= 1 \Rightarrow$ komputer odczyta **-106**, mimo że dodawaliśmy liczby dodatnie!

Część ułamkowa: dziesiętny $\rightarrow$ dwójkowy

Mnożymy ułamek przez 2, zapisujemy część całkowitą wyniku, powtarzamy.

Przykład 10: $0,625_{10} = ?_2$

$$0,625 \cdot 2 = 1,25 \rightarrow 1$$

$$0,25 \cdot 2 = 0,5 \rightarrow 0$$

$$0,5 \cdot 2 = 1,0 \rightarrow 1, \text{ koniec}$$

Wynik: **0,101₂** (sprawdzenie: $1/2 + 1/8 = 0,625 \checkmark$)

Pytanie: Co zwróci w Pythonie wyrażenie `0.1+0.2==0.3`? Dlaczego?

Problem nieskończonych rozwinięć

Przykład 11: $0,1_{10} = ?_2$

$$0,1 \cdot 2 = 0,2 \rightarrow 0$$

$$0,2 \cdot 2 = 0,4 \rightarrow 0$$

$$0,4 \cdot 2 = 0,8 \rightarrow 0$$

$$0,8 \cdot 2 = 1,6 \rightarrow 1$$

$$0,6 \cdot 2 = 1,2 \rightarrow 1$$

$$0,2 \cdot 2 = 0,4 \rightarrow 0 \text{ (powtórka!)}$$

$0,1_{10} = 0,0(0011)_2$ – rozwinięcie **nieskończone i okresowe** $\Rightarrow$ zaokrąglenie.

Odpowiedź: dlaczego w Pythonie `0.1+0.2==0.3` zwraca `False`?

Bo 0,1; 0,2 i 0,3 są przechowywane jako przybliżenia – suma przybliżeń różni się od przybliżenia 0,3.

Konwersja odwrotna: ułamek binarny $\rightarrow$ dziesiętny

Sumujemy wartości ujemnych potęg dwójki tam, gdzie jest cyfra 1.

Przykład 12: $0,1101_2 = ?_{10}$

$$1 \cdot 2^{-1} + 1 \cdot 2^{-2} + 0 \cdot 2^{-3} + 1 \cdot 2^{-4} = 0,5 + 0,25 + 0 + 0,0625 = \mathbf{0,8125}$$

Grupowanie bitów z częścią ułamkową

Ta sama sztuczka grupowania co dla liczb całkowitych – **tylko dopełniamy zerami z prawej** strony (od przecinka na zewnątrz).

Przykład 13: $1010,011_2 = ?_{16} = ?_8$

Hex (grupy po 4): $1010 \rightarrow A$; $011 \rightarrow 0110 \rightarrow 6 \Rightarrow \mathbf{A,6_{16}}$

Oct (grupy po 3): $001\ 010 \rightarrow 1, 2$; $011 \rightarrow 3 \Rightarrow \mathbf{12,3_8}$

(sprawdzenie: $1010,011_2 = 10,375_{10}$; $A,6_{16} = 10 + 6/16$; $12,3_8 = 10 + 3/8 \checkmark$)

Budowa liczby zmiennoprzecinkowej

Liczby rzeczywiste zapisuje się w postaci znormalizowanej, dzieląc bity na trzy pola:

Znak (S)	Wykładnik (E)	Mantysa (M)
1 bit	kilka bitów	pozostałe bity

- **Znak (S):** 0 = dodatnia, 1 = ujemna
- **Wykładnik (E):** przesuwaj przecinek – pełni rolę potęgi podstawy
- **Mantysa (M):** znaczące cyfry, postać $1,xxxx \cdot 2^E$

IEEE 754 (32 bity): 1 bit znaku + 8 bitów wykładnika + 23 bity mantysy. Na maturze liczy się **idea** (skończona mantysa $\Rightarrow$ błędy zaokrągleń), nie pełny format IEEE754.

Ćwiczenia na zajęciach (1/3)

- 1 Dodaj binarnie:
a) $1110 + 0111$ b) $101101 + 011011$
- 2 Odejmij binarnie:
a) $1101 - 0110$ b) $111000 - 010101$
- 3 Zapisz w U2 na 8 bitach:
a) -12 b) -1
- 4 Odczytaj wartość dziesiętną z U2 (8 bit):
a) 11111111 b) 01111111

Ćwiczenia na zajęciach (2/3)

- 5 Zamień na system dwójkowy:
a) $0,75$ b) $0,3125$
- 6 Dodaj/odejmij w systemie szesnastkowym:
a) $4B_{16} + 37_{16}$ b) $80_{16} - 29_{16}$
- 7 Wykonaj przesunięcie bitowe i podaj wynik dziesiętny:
a) $1011_2 \ll 2$ b) $100101_2 \gg 1$
- 8 Zapisz -9 na 8 bitach w ZM, U1 i U2 (porównaj wyniki)

Ćwiczenia na zajęciach (3/3)

- 9 Sprawdź, czy przy dodawaniu na 8 bitach (U2) wystąpi przepełnienie:
a) $90 + 50$ b) $60 + 40$
- 10 Zamień na system dziesiętny:
a) $0,1011_2$ b) $0,111_2$
- 11 Zamień $1101,101_2$ na system szesnastkowy i ósemkowy

- 1 Dodaj binarnie:
a) $1011 + 1110$ b) $11001 + 00111$
- 2 Odejmij binarnie:
a) $10110 - 01011$ b) $11111 - 00001$
- 3 Zapisz w U2 na 8 bitach:
a) -7 b) -100 c) -64
- 4 Odczytaj wartość dziesiętną liczby zapisanej w U2 (8 bit):
a) 11110000 b) 10101010

- 5 Zamień na system dwójkowy:
a) 0,375 b) 0,9 (opisz, co się dzieje przy tej konwersji)
- 6 **(teoretyczne)** Wyjaśnij, dlaczego obliczenia na liczbach zmiennoprzecinkowych bywają "niedokładne"(np. $0.1+0.2 \neq 0.3$)
- 7 Dodaj/odejmij w systemie szesnastkowym:
a) $5E_{16} + 23_{16}$ b) $90_{16} - 48_{16}$
- 8 Wykonaj przesunięcia bitowe i podaj wynik dziesiętny:
a) $1101_2 \ll 3$ b) $111010_2 \gg 2$

- 9 Zapisz -18 na 8 bitach w ZM, U1 i U2
- 10 Sprawdź, czy przy dodawaniu na 8 bitach (U2) wystąpi przepełnienie:
a) $110 + 30$ b) $70 + 40$ c) $(-90) + (-50)$
- 11 Zamień na system dziesiętny:
a) $0,1101_2$ b) $0,0111_2$
- 12 Zamień $10,0101_2$ na system szesnastkowy i ósemkowy

Dziękuję za uwagę!